
# Polly

## Want a 500?

CHAOS ENGINEERING
FOR .NET DEVELOPERS

# Agenda

STORYTIME

THE SCIENCE

THE LIBRARY

THE EXPERIMENT

QUESTIONS

# Hello from Grand Rapids!

I'm Victor Frye

Your friendly neighborhood developer

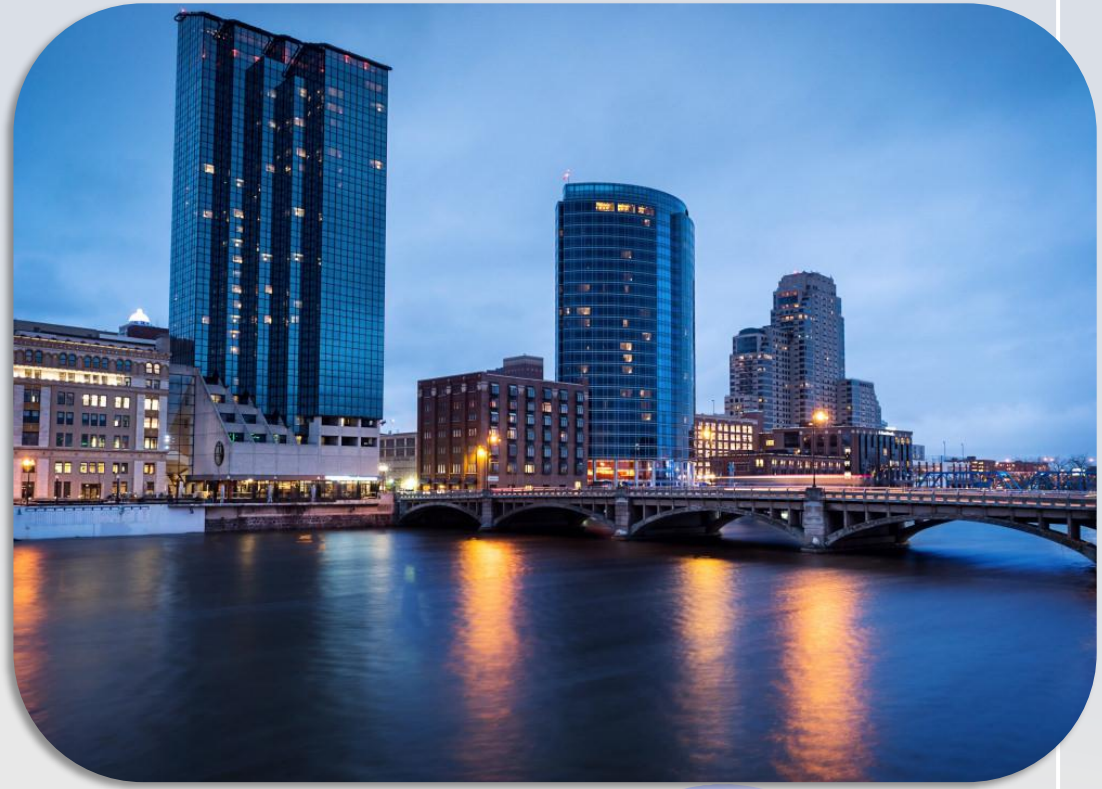
Unchallenged #1 Clippy fan

Love internet culture and content
- victorfrye.com/blog
- microsoftgraveyard.com
- shrugman.com

Passion for .NET, React, Azure, Xbox

DevOps specialist at Leading EDJE

Building with Microsoft solutions



# About **Leading EDJE**

Technology consultancy and services
- Strategists, innovators, partners
- Craft bespoke solutions
- Positive disruption of business norm

Real. Fun. Geeks.

FREE lunch and learns

Expert talent
- Cloud infrastructure
- Digital and app innovation
- Data and AI
- Project management

# The Science

CHAOS ENGINEERING

# **Disaster** stories

1. Email blacklist
2. Unintentional DDOS
3. Mega login
4. Missing documents
5. Cloud outage

# The practice of subjecting workloads to real-world stresses and failures

CHAOS ENGINEERING

# Glossary

| TERM | DEFINITION |
| --- | --- |
| **Availability** | The amount of time that a workload runs in a healthy state |
| **Fault injection** | The act of introducing an error to a system to test resiliency |
| **Recoverability** | The ability to restore normal operations after a disruption within agreed targets |
| **Resiliency** | The ability of a workload to withstand faults and continue operating within an acceptable user experience |

# **Well-Architected** Framework

### Reliability

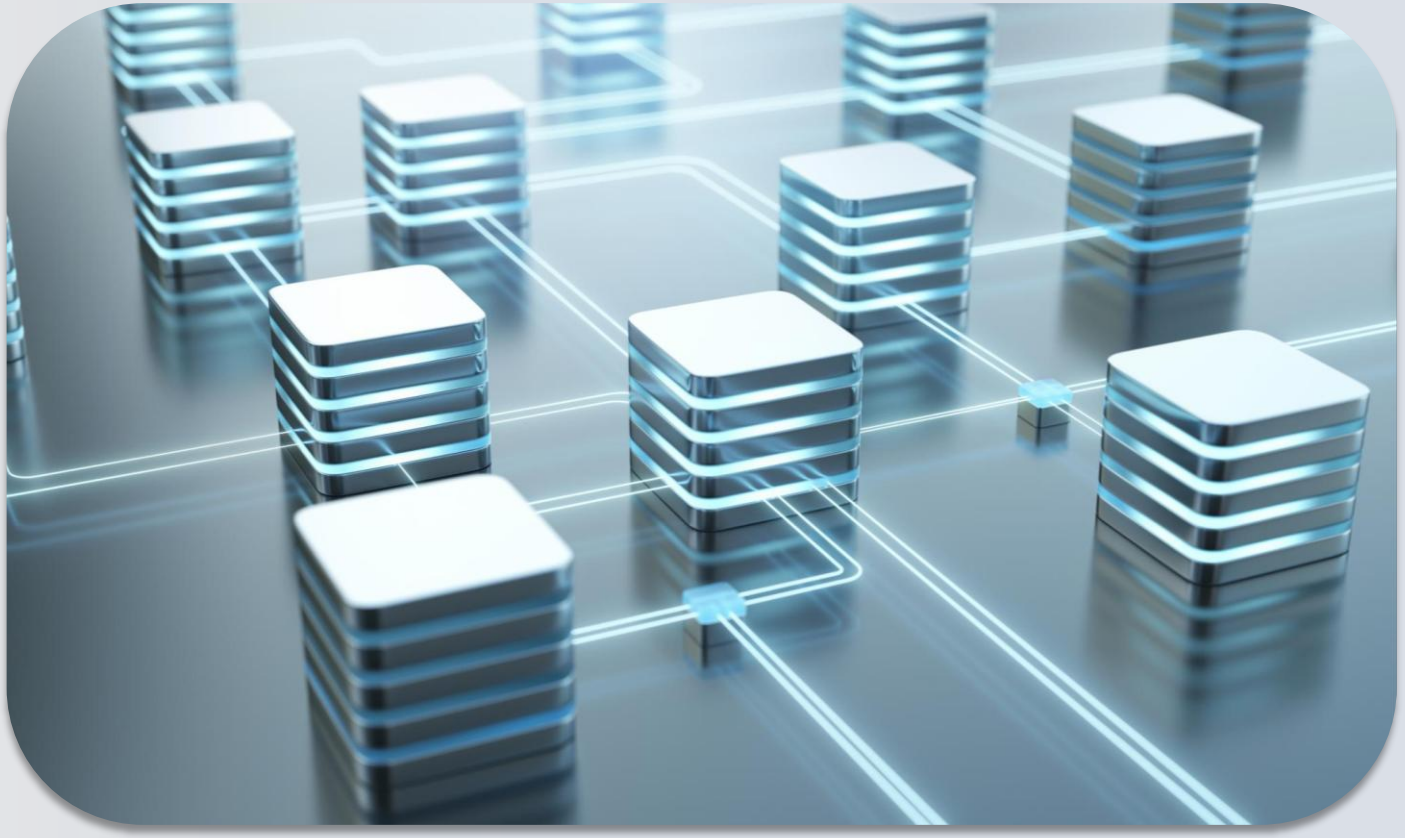
Ensure workloads meet uptime and recovery targets with redundancy and resiliency.

### Operational Excellence

Reduce issues in production by building holistic observability and automated systems.

### Performance Efficiency

Adjust to demand with right sized resources and test changes before deploying to production.



# Chaos science is about conducting experiments to understand system resilience

# The Library

POLLY

# Polly

A .NET resilience library

Open-source

Add resilience pipelines

Works with:
- HttpClient
- TcpClient
- SqlConnection
- File I/O

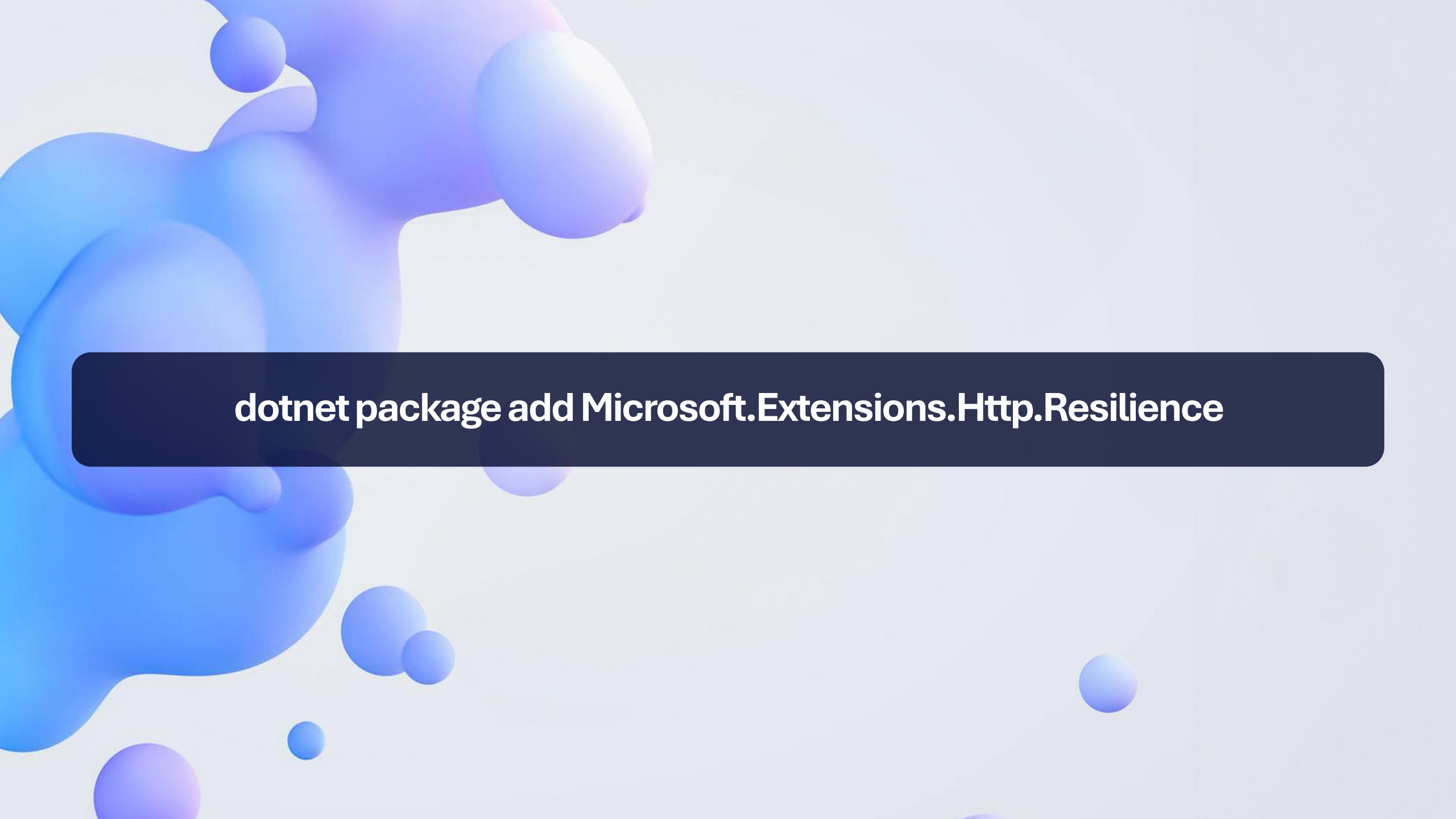
Express strategies:
- Retry
- Circuit-breaker
- Fallback
- Hedging
- Timeout
- Rate limiter
- Chaos

# Resilience Strategies

| STRATEGY | PREMISE | AKA | MITIGATION |
|---|---|---|---|
| **Retry** | Many faults are transient and may self correct after a short delay | *Maybe it's just a blip* | Allow automatic retries |
| **Circuit-breaker** | Failing fast is better than making users/callers wait<br><br>Don't overload an already failing system | *Stop doing it if it hurts*<br><br>*Give that system a break* | Block execution when faults are beyond limits |
| **Fallback** | Things will still fail | *Degrade gracefully* | Define an alternative return value or execution path |
| **Hedging** | Things can be slow sometimes | *Hedge your bets* | Execute parallel actions when things are slow and waits for the fastest one |
| **Timeout** | Can assume failure at a point | *Don't wait forever* | Guaranteee the caller won't have to wait forever |
| **Rate limiter** | Control load by limiting what you accept | *Slow down a bit, will you?* | Constrains exeuctions to not exceeed a certain rate |

# Chaos Strategies

| STRATEGY | ACTION |
| --- | --- |
| **Fault** | Injects exceptions in your system |
| **Outcome** | Injects fake outcomes (results or exceptions) in your system |
| **Latency** | Injects latency into executions before the calls are made |
| **Behavior** | Allows you to inject *any* extra behavior before a call is placed |


dotnet package add Microsoft.Extensions.Http.Resilience

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHttpClient<MyClient>(
    configureClient: static client =>
    {
        client.BaseAddress = new("https://api.victorfrye.com");
    })
    .AddStandardResilienceHandler();
```

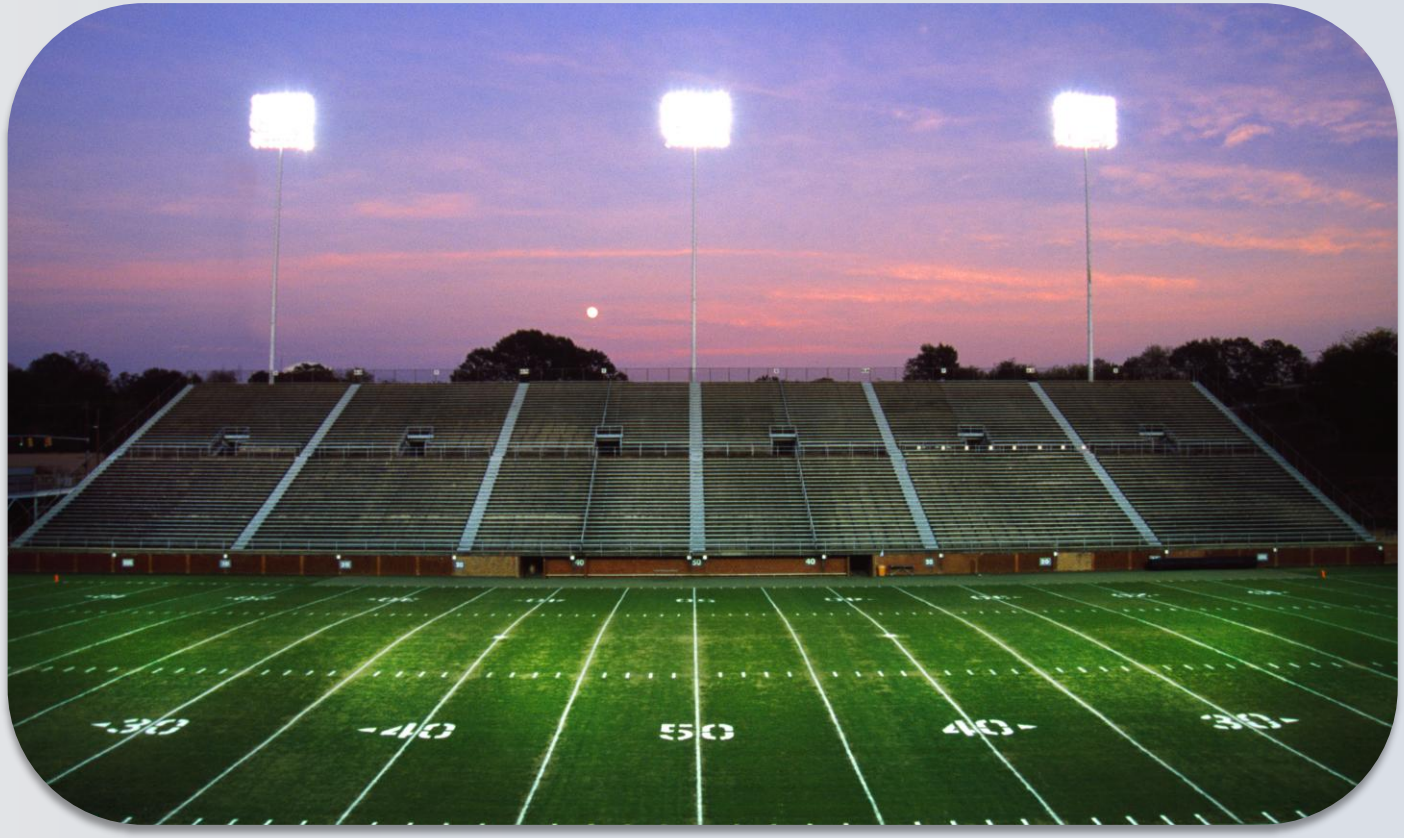
## EXAMPLE CODE FOR ADDING RESILIENCE TO HTTPCLIENT

# Game Day

Prepare for major event

Test for
- Load
- Scalability
- Performance
- Resilience

Install referees



```
dotnet package add Microsoft.FeatureManagement.AspNetCore
```

# The Experiment

READY, SET, GO!

# Thank you


**VICTOR FRYE**
616-706-7407
VICTORFRYE@OUTLOOK.COM
VICTOR.FRYE@LEADINGEDJE.COM
LEADINGEDJE.COM

VICTORFRYE.COM/BLOG
LINKEDIN.COM/IN/VICTORFRYE
GITHUB.COM/VICTORFRYE/PRESENTATIONS